Quick Select

Dr. Tanvir Ahmed

The Selection Problem

- Given an integer k and n unordered elements x_1, x_2, …, x_n, find the kth smallest element in this set. [k should be between 1 to n]
- Example:
  - Input: arr[] = {7, 10, 4, 3, 20, 15}, k = 3 Output: 7 **// 7 is the 3rd smallest item in the list**
  - Input: arr[] = {7, 10, 4, 3, 20, 15} k = 4 Output: 10
- What would be the Naïve solution?
  - One solution is to sort and then pick the kth element
  - Complexity O(n log n) if you use quicksort or merge sort to sort the array
- How can we do this faster?

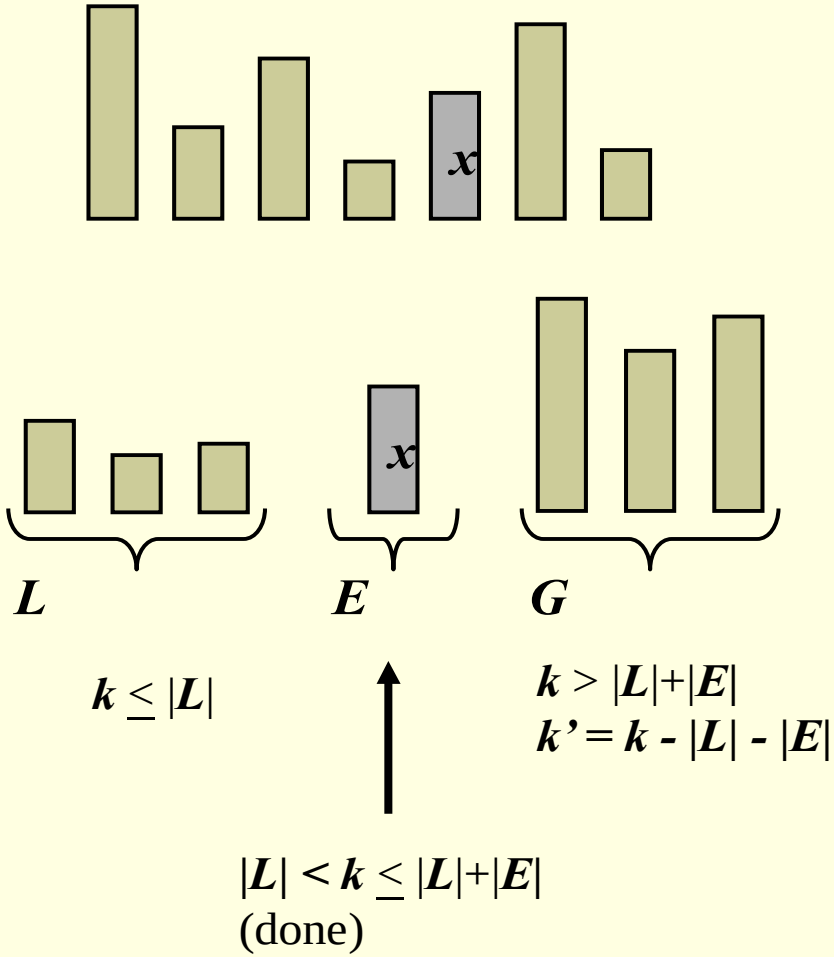
QuickSelect

- Similar to QuickSort with some pruning
- We don’t actually need to sort the entire array
- We only care about finding the kth element
- Main idea:
  - Pick a pivot and partition
  - If k is less than pivot index, look in the left partition

    Else, look in the right partition
  - If k-1 == pivot index, it is the item we are looking for. \\note: k is adjusted for zero–based array

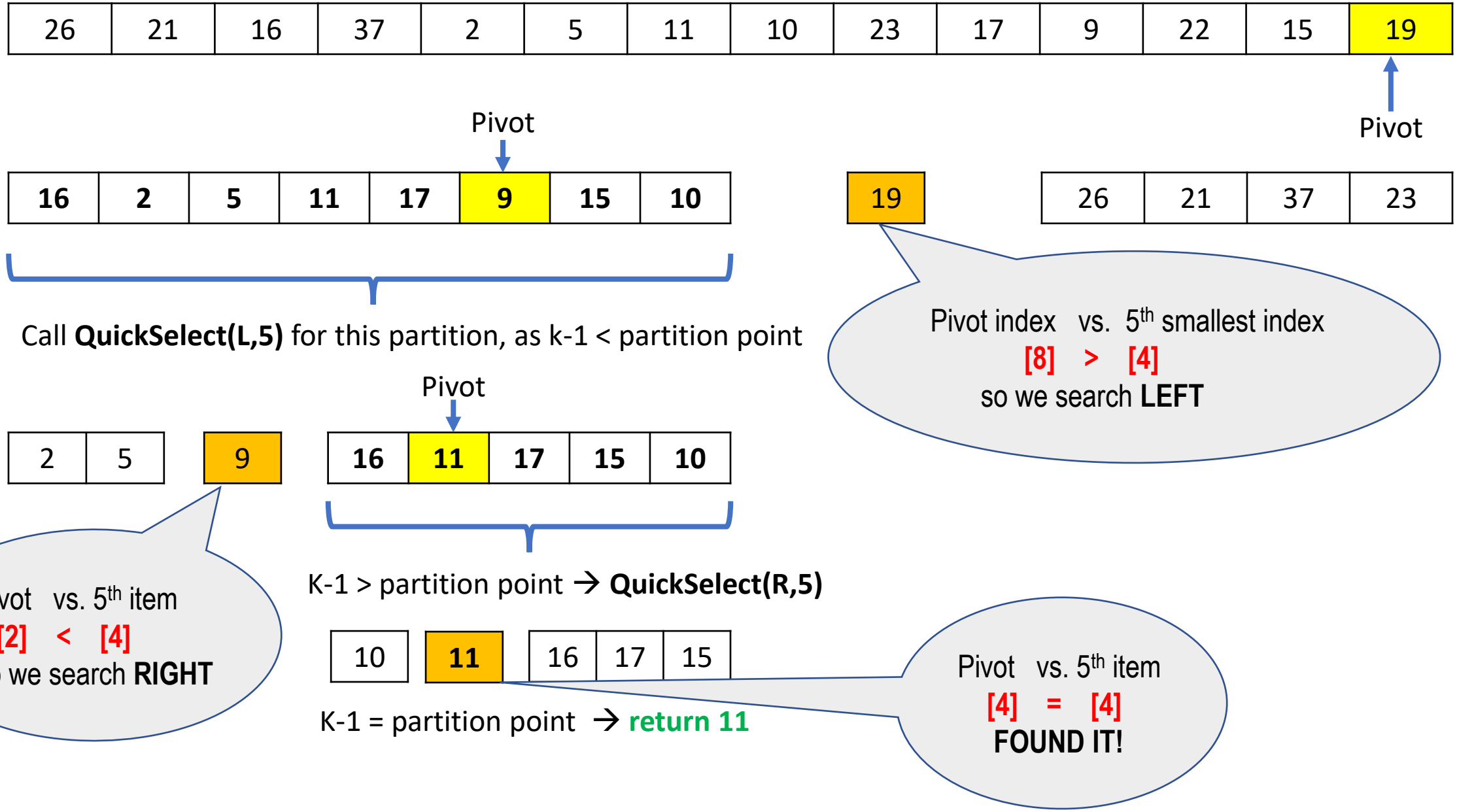
Quick-Select

- Quick-select is a **randomized** selection algorithm based on the prune-and-search paradigm:
  - Prune: pick a random element x (called pivot) and partition S into
    - L elements less than x
    - E elements equal x
    - G elements greater than x
  - Search: depending on k, either answer is in E, or we need to recur on either L or G



Reference: https://www.cs.ucf.edu/courses/cop3502/spr2012/notes/COP3502_28b_QuickSelect.pdf

Example: k = 5


26 21 16 37 2 5 11 10 23 17 9 22 15 19
Pivot
Pivot
16 2 5 11 17 9 15 10
19
26 21 37 23
Call QuickSelect(L,5) for this partition, as k-1 < partition point
Pivot index vs. 5th smallest index
[8] > [4]
so we search LEFT
Pivot
2 5
9
16 11 17 15 10
Pivot vs. 5th item
[2] < [4]
so we search RIGHT
K-1 > partition point → QuickSelect(R,5)
10
11
16 17 15
Pivot vs. 5th item
[4] = [4]
FOUND IT!
K-1 = partition point → return 11

Quick Select Pseudo Code looks something like this

```
function quickSelect(array, left, right, k)

   if (left == right)
      return array[left] //we got one item, so this has to be it

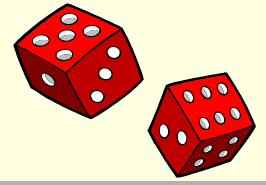
   pivotIndex = partition(array, left, right)
   if (k-1 == pivotIndex)
      return array[pivotIndex] //we found k-th smallest
   else if (pivotIndex > k-1 ) // pivot is n-th > k-th  (further in ordering)
      quickSelect(array, left, pivotIndex - 1, k)   //search left
   else // pivot is n-th < k-th (earlier in ordering)
      quickSelect(array, pivotIndex + 1, right, k) //search right
```

TODO: Use the same partition method we learned in quick sort and try to implement QuickSelect in C

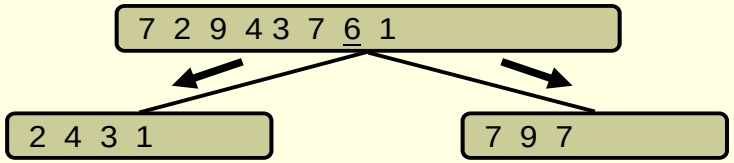
Time Complexity

- We already learned about the run time of quicksort
  - Best-case O(n log n) if always we get middle value as pivot
  - Worst-case $O(n^2)$, if we get smallest/largest as pivot
  - Average-case O(n log n) if sometimes we get middle, sometimes smallest/largest as pivot
- Like quicksort, quickselect has a good average performance, but sensitive to the chosen pivot.
- A good pivot at every step can significantly decrease the search space.
- In QuickSelect, the complexity can be reduced to O(n) from O(n log n), with a worst case of $O(n^2)$
- The worst case occurs for example searching for the maximum element of a set, using first element as the pivot.

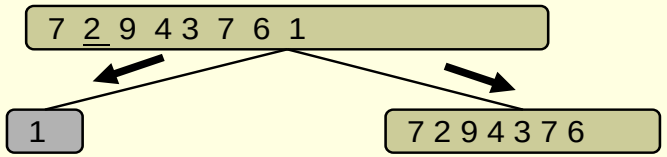
Running Time



- Best Case - even splits (n/2 and n/2)
- Worst Case - bad splits (1 and n-1)



Good call



Bad call

<u>Best case:</u> we choose k-th element as our first pivot

- We look through all n
- First pivot's final position is k-1 and we return → **O(n)**

<u>Worst case:</u> we choose the largest or smallest every time

- We look through all n on first iteration;
  next iteration n-1
  next iteration n-2
  …
- n + (n-1) + (n-2) + … + 1 = n*(n+1)/2 → **$O(n^2)$**

<u>Average case</u>: we remove about half each time (sometimes more, sometimes less)

- N + n/2 + n/4 + … + 1 = 2n - 1 → **O(n)**

Reference: https://www.cs.ucf.edu/courses/cop3502/spr2012/notes/COP3502_28b_QuickSelect.pdf

Exercise

- You already have the pseudo code
- You already have the partition function from quick sort.
- Write the complete program for Quick select.